Push notifications: step-by-step build guide (v2)

Nov 3, 2026 · @danial

Overview

By the end of this guide, a user who taps **submit form b** sends a push notification to every Consultant in the same package, on Android and iPhone. Follow the parts in order: A, B, C, D, then E to test on Android. Parts F and G add the iPhone.

What you will have built

- A bell with a badge in the Form Depot tab header. Tapping it opens a Notifications page.
- On that page, a **submit form b** button. It notifies only users whose login `m_userType` is `Consultant` and whose `m_userPackage` equals the submitter's package.
- At login, each phone tells your server "this token belongs to this user, of this type, in this package". At logout, it removes that link.
- A small server (Express + Postgres) that stores the tokens and the notifications, and asks Firebase to deliver the push.

Who receives a notification

The app sends the submitter's package with the request. The server then selects the phones where both rules hold:

Rule	Source in the login response
User type is `Consultant`	`user_profile_data.m_userType`
Package equals the submitter's package	`user_profile_data.m_userPackage`

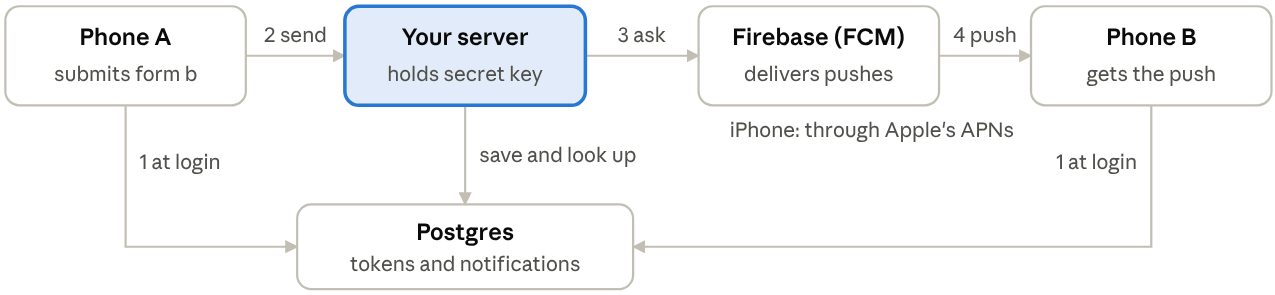
Example: user `user.level4` has type `Consultant` and package `3`. A submitter in package 3 notifies this user. A submitter in package 4 does not.

How a notification travels (phone A submits, phone B receives)

1. Both phones log in. Each one gets its Firebase token and sends it to your server with the username, user type and package. The server saves them in the database.

2. On phone A you tap **submit form b**. The app calls your server with the target type (`Consultant`) and phone A's package.
3. The server finds every token with that type and package, saves one notification row per recipient user, and asks Firebase to deliver.
4. Firebase delivers the push to the phones: directly on Android, through Apple's push service (APNs) on iPhone. If a token is dead (for example the app was reinstalled), the server deletes it.

Your server, not the app, asks Firebase to deliver the push



push flow · 4 steps, your server in the middle

Step 1 is the login on both phones. Steps 2 to 4 are the submit. On an iPhone, Firebase hands the push to Apple's push service (APNs).

The app never sends through Firebase itself. Only your server holds the Firebase secret key.

Files that change, and what you need first

The notification feature touches nine files inside `lib` , a few files outside `lib` , a new server folder and, for the iPhone, a few iOS files.

Inside `lib`

File	Change	Where in this guide
`lib/firebase_options.dart`	New, generated by a tool	Part A, step A3
`lib/constant/fcm_config.dart`	New	Part D, step 1

File	Change	Where in this guide
`lib/noti_services/push_service.dart`	New	Part D, step 2
`lib/screen/notification/notification_screen.dart`	New	Part D, step 7
`lib/main.dart`	Edited	Part D, step 3
`lib/controller/auth/auth_controller.dart`	Edited	Part D, step 4
`lib/controller/home/root_home_controller.dart`	Edited	Part D, step 5
`lib/constant/routes.dart`	Edited	Part D, step 6
`lib/screen/home/root_home_screen.dart`	Edited	Part D, step 8

Outside `lib`

File or place	Change	Where in this guide
`pubspec.yaml`	Two packages added	Part A, step A4
`android/app/google-services.json`	New, generated by a tool	Part A, step A3
`android/settings.gradle`, `android/app/build.gradle`	Check the Google services plugin	Part A, step A5
`android/gradle.properties`	More memory for Gradle	Part A, step A6
`android/app/src/main/AndroidManifest.xml`	Allow plain `http` (testing only)	Part A, step A7
`fcm_server` folder (new, outside your Flutter project)	Database and Express server	Parts B and C
`ios/Runner/GoogleService-Info.plist`	New, generated by a tool	Part F, step F4

File or place	Change	Where in this guide
`ios/Runner/Info.plist`	Plain `http` and local network entries	Part F, step F9
Xcode project settings	Signing, Push Notifications, Background Modes	Part F, step F8

Nothing else inside `lib` is touched. In the code below the app's package name is `tram`, so imports look like `package:tram/...`. If your `pubspec.yaml` `name:` is different, use that name.

Before you start

- Flutter working, Node.js (LTS), Docker (or a local Postgres), a Google account and an Android phone.
- A second device with a second account, only for the final test.
- For the iPhone (Parts F and G): a **Mac** with Xcode (Flutter cannot build iOS on Windows), a **paid Apple Developer account**, and a real iPhone. The iOS simulator cannot receive pushes from Firebase.
- The PC that runs the server and the phones must be on the same Wi-Fi.
- Parts A to E use Windows PowerShell on the PC. Part F uses Terminal on the Mac.

Part A: Setup outside lib

Part A connects your app to Firebase, Google's push service. Run every command in your app's root folder (the one that contains `lib`, `pubspec.yaml` and `android`) unless a step says "anywhere". These steps use Windows PowerShell.

A1. Create a Firebase project

1. Go to console.firebase.google.com and sign in with a Google account.
2. Click **Create a project**, name it (ours is `trams-test`, project id `trams-test-f2a06`), turn Google Analytics off, and create it.
3. This is a test project. When the app goes to production, your organisation's own Firebase project replaces it: repeat A3 and the Firebase key step in Part C against that project.

A2. Install the tools (anywhere)

Install the LTS version of Node.js from nodejs.org if `node -v` says "not recognized". Then run:

```
npm install -g firebase-tools
firebase login
dart pub global activate flutterfire_cli
```

Close and reopen the terminal after the first command, or `firebase` will not be found. After the third command, Windows warns that `C:\Users\User\AppData\Local\Pub\Cache\bin` is not on your PATH. Add that folder to your user PATH (Windows search, "Edit the system environment variables", Environment Variables, User variables, Path, New), then reopen the terminal. For one window only, this works instead:

```
$env:Path += ";C:\Users\User\AppData\Local\Pub\Cache\bin"
```

A3. Connect the app to Firebase

```
flutterfire configure --project=trams-test-f2a06 --platforms=android
```

On the PC we configure Android only. Never run the command without `--platforms`: it selects every platform and the web registration fails. This command creates two files: `lib/firebase_options.dart` and `android/app/google-services.json`. It also registers the Android app `com.ai.trams` in your Firebase project. Never type these two files by hand.

The iPhone is added later, in Part F step F4, by running the same command on the Mac with `--platforms=android,ios`. That run regenerates `lib/firebase_options.dart` with iOS values.

A4. Add the packages

```
flutter pub add firebase_core firebase_messaging
```

This adds two lines under `dependencies:` in `pubspec.yaml`. The app already uses `dio` (for network calls), `get` and `get_storage`, so nothing else is needed.

A5. Check the Android build files

Firebase needs the `google-services` plugin in two files. Check that both have it:

```
findstr /s /i "google-services" android\*.gradle android\*.kts
```

In our project both already had it:

- `android/settings.gradle`, in the `plugins` block: `id "com.google.gms.google-services" version "4.3.15" apply false`
- `android/app/build.gradle`, in the `plugins` block: `id 'com.google.gms.google-services'`

If either is missing, add that line. If the build later complains about `minSdkVersion`, set it to 23 in `android/app/build.gradle`.

A6. Give Gradle more memory

Our first build with Firebase failed with "Java heap space". Open `android/gradle.properties`, find the `org.gradle.jvmargs=` line (add it if missing), and set:

```
org.gradle.jvmargs=-Xmx4G -XX:MaxMetaspaceSize=2G
```

Use `-Xmx3G` on a PC with 8 GB of RAM. Then stop the old Gradle process so the new value is used:

```
cd android
.\gradlew --stop
cd ..
```

Only if the same heap error still appears, also set `android.enableJetifier=false` in the same file. Warnings that Gradle, the Android Gradle Plugin or Kotlin versions will "soon be dropped" are harmless.

A7. Allow plain `http` on Android (testing only)

In the test your phone calls the server on your PC at an address like `http://192.168.0.113:4000`. Android blocks plain `http` by default. Open `android/app/src/main/AndroidManifest.xml` and add one attribute to the `<application ...>` tag:

```
<application
    android:usesCleartextTraffic="true"
    ...keep the attributes that are already there...>
```

This is for testing only. A production server uses `https`, and this line is then removed. The iPhone needs its own version of this, in Part F step F9.

Part B: The database

The server needs two tables: one remembers which phone belongs to which user (with that user's type and package), the other stores the notifications. We use PostgreSQL in Docker, on port **5004**. If you already have a Postgres you can use, skip B1 and run the SQL from B2 in a database called `trams_noti_db`.

B1. Start a Postgres database

Create a folder for the notification server, for example `fcm_server`, outside your Flutter project. Inside it create `docker-compose.yml`:

```
services:
  postgres:
    image: postgres:16-alpine
    container_name: trams_noti_db
    restart: unless-stopped

    environment:
      POSTGRES_DB: trams_noti_db
      POSTGRES_USER: root
      POSTGRES_PASSWORD: root

    ports:
      - 5004:5432

    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

The user and password here (`root` / `root`) are for local testing. Use a stronger password for anything shared, and remember it: Part C needs it. Start the database from that folder:

```
docker compose up -d
```

The database now listens on your PC at port **5004** (the number on the left of `5004:5432`). Remember the user, password, port and database name: Part C needs them.

B2. Create the two tables

In the same folder create `schema.sql`:

```
CREATE TABLE IF NOT EXISTS device_tokens (
  id            SERIAL PRIMARY KEY,
  username      TEXT NOT NULL,
  fcm_token     TEXT NOT NULL UNIQUE,
  platform      TEXT NOT NULL DEFAULT 'android',
  user_type     TEXT,
  user_package  TEXT,
  updated_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_device_tokens_username ON device_tokens
(username);
CREATE INDEX IF NOT EXISTS idx_device_tokens_type_package ON device_tokens
(user_type, user_package);

CREATE TABLE IF NOT EXISTS notifications (
  id            SERIAL PRIMARY KEY,
  to_username   TEXT NOT NULL,
  from_username TEXT,
  title         TEXT NOT NULL,
  body          TEXT NOT NULL DEFAULT '',
  is_read       BOOLEAN NOT NULL DEFAULT false,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);
CREATE INDEX IF NOT EXISTS idx_notifications_user ON notifications
(to_username, is_read);
```

Run it:

```
Get-Content schema.sql | docker compose exec -T postgres psql -U root -d
trams_noti_db
```

You should see `CREATE TABLE` twice and `CREATE INDEX` three times. If you use a database tool such as DBeaver or the VS Code database extension instead, connect to `localhost`, port `5004`, and paste the SQL into a query window.

If you created `device_tokens` earlier without the two new columns, add them instead of recreating the table:

```
docker exec -it trams_noti_db psql -U root -d trams_noti_db -c "ALTER TABLE
device_tokens ADD COLUMN IF NOT EXISTS user_type TEXT;"
docker exec -it trams_noti_db psql -U root -d trams_noti_db -c "ALTER TABLE
device_tokens ADD COLUMN IF NOT EXISTS user_package TEXT;"
```

B3. Check it worked

Run these two queries in your database tool. Both should return zero rows and no error:

```
SELECT * FROM device_tokens;
SELECT * FROM notifications;
```

To confirm the two new columns exist, run:

```
docker exec -it trams_noti_db psql -U root -d trams_noti_db -c "\d
device_tokens"
```

You should see `user_type` and `user_package` in the list.

What the tables hold

Table	One row means	Key rule
`device_tokens`	One phone (one app install) that is logged in as a user	`fcm_token` is `UNIQUE`, so a token belongs to only one user at a time. If another user logs in on the same phone, the row moves to that user, including the new `user_type` and `user_package`.
`notifications`	One notification sent to one user	`is_read` starts `false`. The bell count and the notifications list will read from this table.

We use `username` (the value the app already gets as `mUsername` at login) to identify users, because the app has no other user id. `user_type` is the login response's `m_userType` (for example `Consultant`) and `user_package` is `m_userPackage` (for example `3`), both saved as text.

Part C: The notification server

The app cannot send pushes itself, because sending needs a secret key from your Firebase project. A small Express server holds that key, remembers which phone belongs to which user, and asks Firebase to deliver. It lives in the `fcm_server` folder from Part B, outside your Flutter project. Every file below uses ES module style (`import`, not `require`).

C1. Get the Firebase secret key

1. In the Firebase console, open the `trams-test` project, then **Settings** (gear icon) and **Project settings**.
2. Open the **Service accounts** tab and click **Generate new private key**.
3. Save the downloaded file as `serviceAccountKey.json` inside `fcm_server`.

This file is a secret. Never put it in the Flutter app, share it, upload it to GitHub, or send it in a zip. If it ever leaves your machine, generate a new key and delete the old one in the same tab.

C2. Create the Node project

Run these inside `fcm_server`:

```
npm init -y
npm install express firebase-admin cors pg dotenv
```

Then open the new `package.json` and add one line inside the top-level braces, so Node understands `import`:

```
"type": "module",
```

C3. Secrets file and .gitignore

Create `.env` in `fcm_server`. Use the user, password, port and database name from Part B:

```
DATABASE_URL=postgres://root:root@localhost:5004/trams_noti_db
```

If your password contains characters such as `@` or `#`, they must be URL-encoded in this line. Easiest is to use a password with only letters and numbers.

Create `.gitignore` in the same folder, so secrets never reach GitHub:

```
.env
serviceAccountKey.json
node_modules
```

C4. db.js (the database connection)

```
// One shared connection pool for the whole server.
import 'dotenv/config';
import pg from 'pg';

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL });

export default pool;
```

C5. auth.js (who is calling?)

Every endpoint that needs to know the user uses this one function. For now it reads the username from a request header. This is a **test stand-in**: in production, this is the only place you change, to verify the real access token from your login API and set `req.user` from it. Everything else in the server uses `req.user.username`.

```
// TEST STAND-IN for authentication.
// Production: verify the real access token here and set req.user from it.
// Everything else in the server just uses req.user.username.
export function requireUser(req, res, next) {
  const username = req.get('X-Test-User');
  if (!username) {
    return res.status(401).json({ ok: false, error: 'not logged in' });
  }
  req.user = { username };
  next();
}
```

C6. server.js (the whole file)

This version has everything: it stores each phone's user type and package, sends to one user by username, and sends to a whole user type within one package.

```
// Notification server: stores device tokens and notifications,
// and asks Firebase (FCM) to deliver pushes.
// Needs: serviceAccountKey.json (from Firebase console) in the same folder.

import express from 'express';
import cors from 'cors';
import { readFileSync } from 'fs';
import { initializeApp, cert } from 'firebase-admin/app';
import { getMessaging } from 'firebase-admin/messaging';
import pool from './db.js';
import { requireUser } from './auth.js';

// ES modules can't `require` a .json file, so read and parse it instead.
const serviceAccount = JSON.parse(
  readFileSync(new URL('./serviceAccountKey.json', import.meta.url), 'utf8')
);

initializeApp({
  credential: cert(serviceAccount),
});

const app = express();
app.use(cors());
app.use(express.json());

// Quick check that the server is up: open http://localhost:4000 in a
browser.
app.get('/', (req, res) => {
  res.send('FCM server is running');
});

// Checks the server can talk to the database.
app.get('/health', async (req, res) => {
  try {
    const r = await pool.query('SELECT COUNT(*) FROM device_tokens');
    res.json({ ok: true, deviceTokens: Number(r.rows[0].count) });
  } catch (err) {
    console.error('DB check failed:', err.message);
```

```
    res.status(500).json({ ok: false, error: err.message });
  }
});

// POST /send-test  (simple test, no database, no login)
// Body (all optional):
//   { "token": "...", "topic": "all_users", "title": "...", "body": "..." }
// If "token" is given, sends to that one phone. Otherwise sends to the
topic.
app.post('/send-test', async (req, res) => {
  const {
    token,
    topic = 'all_users',
    title = 'Test notification',
    body = 'Hello from the Express server',
  } = req.body || {};

  const message = {
    notification: { title, body },
    // high priority = Android delivers right away instead of waiting
    android: { priority: 'high' },
    ...(token ? { token } : { topic }),
  };

  try {
    const id = await getMessaging().send(message);
    console.log('Sent:', id, token ? '(to token)' : `(to topic ${topic})`);
    res.json({ ok: true, id });
  } catch (err) {
    console.error('Send failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// POST /devices
// body: { "token": "...", "platform": "android", "userType": "Consultant",
"userPackage": "3" }
// Called by the app at login. Links this phone's token to the logged-in
user,
// and remembers that user's type and package.
// If the token already exists (same phone, different user), it moves to this
user.
app.post('/devices', requireUser, async (req, res) => {
```

```
  const { token, platform = 'android', userType = null, userPackage = null }
= req.body || {};
  if (!token) {
    return res.status(400).json({ ok: false, error: 'token required' });
  }
  try {
    await pool.query(
      `INSERT INTO device_tokens (username, fcm_token, platform, user_type,
user_package)
       VALUES ($1, $2, $3, $4, $5)
       ON CONFLICT (fcm_token)
       DO UPDATE SET username = EXCLUDED.username,
                     platform = EXCLUDED.platform,
                     user_type = EXCLUDED.user_type,
                     user_package = EXCLUDED.user_package,
                     updated_at = now()`,
      [req.user.username, token, platform, userType, userPackage]
    );
    console.log('Registered device for', req.user.username);
    res.json({ ok: true });
  } catch (err) {
    console.error('Register failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// DELETE /devices   body: { "token": "..." }
// Called by the app at logout. Only removes the link if it belongs to this
user.
app.delete('/devices', requireUser, async (req, res) => {
  const { token } = req.body || {};
  if (!token) {
    return res.status(400).json({ ok: false, error: 'token required' });
  }
  try {
    const r = await pool.query(
      'DELETE FROM device_tokens WHERE fcm_token = $1 AND username = $2',
      [token, req.user.username]
    );
    console.log('Unregistered device for', req.user.username, '- rows:',
r.rowCount);
    res.json({ ok: true, deleted: r.rowCount });
  } catch (err) {
```

```
    console.error('Unregister failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// POST /notifications   body: { "toUsername": "...", "title": "...", "body":
"..." }
// Sends to one user by username (not used by the Notifications page any
more,
// but kept for tests and later features).
// The sender is whoever is logged in (req.user), never taken from the body.
app.post('/notifications', requireUser, async (req, res) => {
  const { toUsername, title, body = '' } = req.body || {};
  if (!toUsername || !title) {
    return res.status(400).json({ ok: false, error: 'toUsername and title
required' });
  }

  try {
    // 1. Save it, so the bell count and notifications page can show it
later.
    const saved = await pool.query(
      `INSERT INTO notifications (to_username, from_username, title, body)
       VALUES ($1, $2, $3, $4)
       RETURNING id`,
      [toUsername, req.user.username, title, body]
    );
    const notificationId = saved.rows[0].id;

    // 2. Find every phone this user is logged in on.
    const devices = await pool.query(
      'SELECT fcm_token FROM device_tokens WHERE username = $1',
      [toUsername]
    );
    const tokens = devices.rows.map((r) => r.fcm_token);

    // 3. Send to all of them.
    let sent = 0;
    let removed = 0;
    if (tokens.length > 0) {
      const result = await getMessaging().sendEachForMulticast({
        tokens,
        notification: { title, body },
```

```
        android: { priority: 'high' },
        apns: { payload: { aps: { sound: 'default' } } },
        data: { notificationId: String(notificationId) },
      });
      sent = result.successCount;

      // 4. Clean up tokens Firebase says are dead (e.g. app was
reinstalled).
      const dead = [];
      result.responses.forEach((r, i) => {
        const code = r.error?.code;
        if (
          !r.success &&
          (code === 'messaging/registration-token-not-registered' ||
            code === 'messaging/invalid-registration-token')
        ) {
          dead.push(tokens[i]);
        }
      });
      if (dead.length > 0) {
        await pool.query(
          'DELETE FROM device_tokens WHERE fcm_token = ANY($1)',
          [dead]
        );
        removed = dead.length;
      }
    }

    console.log(
      `Notification ${notificationId}: ${req.user.username} -> ${toUsername}
(devices ${tokens.length}, sent ${sent}, removed ${removed})`
    );
    res.json({ ok: true, id: notificationId, devices: tokens.length, sent,
removed });
  } catch (err) {
    console.error('Notify failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

// POST /notifications/by-type
// body: { "toUserType": "Consultant", "toUserPackage": "3", "title": "...",
"body": "..." }
```

```
// Sends to every phone whose logged-in user has this user type AND this
package.
// One notifications row is saved per recipient username, so the bell list
can use it later.
app.post('/notifications/by-type', requireUser, async (req, res) => {
  const { toUserType, toUserPackage, title, body = '' } = req.body || {};
  if (!toUserType || !toUserPackage || !title) {
    return res
      .status(400)
      .json({ ok: false, error: 'toUserType, toUserPackage and title
required' });
  }

  try {
    // 1. Find every phone logged in as this user type, in this package.
    const devices = await pool.query(
      'SELECT username, fcm_token FROM device_tokens WHERE user_type = $1 AND
user_package = $2',
      [toUserType, String(toUserPackage)]
    );
    const tokens = devices.rows.map((r) => r.fcm_token);
    const usernames = [...new Set(devices.rows.map((r) => r.username))];

    // 2. Save one notification per recipient user.
    for (const u of usernames) {
      await pool.query(
        `INSERT INTO notifications (to_username, from_username, title, body)
         VALUES ($1, $2, $3, $4)`,
        [u, req.user.username, title, body]
      );
    }

    // 3. Send to all of the phones.
    let sent = 0;
    let removed = 0;
    if (tokens.length > 0) {
      const result = await getMessaging().sendEachForMulticast({
        tokens,
        notification: { title, body },
        android: { priority: 'high' },
        apns: { payload: { aps: { sound: 'default' } } },
      });
      sent = result.successCount;
```

```
      // 4. Clean up dead tokens.
      const dead = [];
      result.responses.forEach((r, i) => {
        const code = r.error?.code;
        if (
          !r.success &&
          (code === 'messaging/registration-token-not-registered' ||
            code === 'messaging/invalid-registration-token')
        ) {
          dead.push(tokens[i]);
        }
      });
      if (dead.length > 0) {
        await pool.query('DELETE FROM device_tokens WHERE fcm_token =
ANY($1)', [dead]);
        removed = dead.length;
      }
    }

    console.log(
      `Notification by type "${toUserType}" pkg ${toUserPackage}:
${req.user.username} -> ${usernames.length} user(s) (devices
${tokens.length}, sent ${sent}, removed ${removed})`
    );
    res.json({ ok: true, users: usernames.length, devices: tokens.length,
sent, removed });
  } catch (err) {
    console.error('Notify by type failed:', err.message);
    res.status(500).json({ ok: false, error: err.message });
  }
});

const PORT = 4000;
// '0.0.0.0' lets your phone reach this server over Wi-Fi, not just this PC.
app.listen(PORT, '0.0.0.0', () => {
  console.log(`FCM server listening on port ${PORT}`);
});
```

The app sends the submitter's package in `toUserPackage`, and the server trusts it. That is fine for testing. When you replace `auth.js` with real token checking, the server should read the submitter's package from the verified login instead of from the request.

C7. Run it and test without the app

Start the server. If Windows shows a firewall popup, allow it on **Private networks**.

```
node server.js
```

It prints `FCM server listening on port 4000`. Restart it (Ctrl+C, then `node server.js` again) every time you edit `server.js`. Then check each piece from a second PowerShell window.

1. Database link. Open `http://localhost:4000/health` in a browser. Expected: `{"ok":true,"deviceTokens":0}`.

2. Two accounts on one phone. Pretend Ali then Siti log in on the same phone (same token):

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "ali" } -ContentType "application/json" -Body
'{"token":"fake-token-123","userType":"Consultant","userPackage":"3"}'
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "siti" } -ContentType "application/json" -Body
'{"token":"fake-token-123","userType":"Consultant","userPackage":"3"}'
```

Run `SELECT username, user_type, user_package, fcm_token FROM device_tokens;`. Expected: **one** row, `siti`, `Consultant`, `3`. The token moved from Ali to Siti.

3. Only the owner can unregister. Ali tries to delete Siti's link:

```
Invoke-RestMethod -Method Delete -Uri http://localhost:4000/devices -Headers
@{ "X-Test-User" = "ali" } -ContentType "application/json" -Body
'{"token":"fake-token-123"}'
```

Expected: `deleted 0`. Repeat with `"siti"` and you get `deleted 1`.

4. A notification for a user with no phone.

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/notifications -
Headers @{ "X-Test-User" = "tester" } -ContentType "application/json" -Body
'{"toUsername":"nobody","title":"Hello","body":"From tester"}'
```

Expected: `devices 0, sent 0`, but the row is saved in `notifications`. Delivery to a real phone is tested in Part E.

5. Consultants in the same package only. Register three fake phones: a Consultant in package 3, a Consultant in package 4, and a Service Provider in package 3:

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "cons3" } -ContentType "application/json" -Body
'{"token":"fake-c3","userType":"Consultant","userPackage":"3"}'
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "cons4" } -ContentType "application/json" -Body
'{"token":"fake-c4","userType":"Consultant","userPackage":"4"}'
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "sp3" } -ContentType "application/json" -Body
'{"token":"fake-s3","userType":"Service Provider","userPackage":"3"}'
```

Now send to Consultants in package 3:

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/notifications/by-
type -Headers @{ "X-Test-User" = "submitter" } -ContentType
"application/json" -Body
'{"toUserType":"Consultant","toUserPackage":"3","title":"Form B
submitted","body":"test"}'
```

Make sure the `siti` row from step 3 is gone first (the delete with `siti` returned `deleted 1`). If it is still there, `siti` is also a Consultant in package 3 and you will see `devices 2`.

Expected: `users 1, devices 1`, so only `cons3` matched. Firebase rejects the fake token, so you will most likely also see `sent 0, removed 1`. Run `SELECT to_username, from_username, title FROM notifications ORDER BY id DESC;`: the newest row is for `cons3` only. Sending without `toUserPackage` returns the error `toUserType, toUserPackage and title required`.

Part D: Changes inside lib

This part edits your original `lib` folder. Do steps 1 to 8 in order. `flutter analyze` will show errors until all eight are done, because the files refer to each other. Run it after step 8.

Every "Before" block below is copied from the original `lib`. If your file does not match, stop and compare before editing. The code already includes the iPhone parts (the iOS token handling and the permission request at login), so you only write each file once. They do nothing on Android.

Step 1. New file `lib/constant/fcm_config.dart`

This holds the address of your notification server. Create the file with:

```
// Address of the notification server (fcm_server).
// Run `ipconfig` on your PC and use its IPv4 address. Phone and PC must be
on the same Wi-Fi.
// Keep the :4000 at the end.
const String fcmServerUrl = 'http://192.168.0.113:4000';
```

Change `192.168.0.113` to your PC's IPv4 address. It can change when you reconnect to Wi-Fi, so check it again if the app suddenly cannot reach the server.

Step 2. New file `lib/noti_services/push_service.dart`

Create the folder `lib/noti_services/` and put this file in it. Every call the app makes to the notification server goes through this one file. The folder name matters: the imports in later steps must match it exactly, or you get "Target of URI doesn't exist".

```
import 'package:dio/dio.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/foundation.dart';
import 'package:tram/constant/fcm_config.dart';

class PushService {
  static final Dio _dio = Dio(BaseOptions(
    baseUrl: fcmServerUrl,
    connectTimeout: const Duration(seconds: 5),
    receiveTimeout: const Duration(seconds: 5),
  ));

  // TEST STAND-IN. Matches requireUser() in the server's auth.js.
  // Production: send the real access token here, e.g. {'Authorization':
'Bearer $accessToken'}.
  static Map<String, String> _authHeaders(String username) =>
      {'X-Test-User': username};

  /// On iOS the FCM token only works once Apple's token is ready.
  static Future<String?> _getFcmToken() async {
    if (defaultTargetPlatform == TargetPlatform.iOS) {
      String? apns;
      for (var i = 0; i < 10 && apns == null; i++) {
        apns = await FirebaseMessaging.instance.getAPNSToken();
```

```
        if (apns == null) {
          await Future.delayed(const Duration(milliseconds: 500));
        }
      }
      if (apns == null) return null;
    }
    return FirebaseMessaging.instance.getToken();
  }

  /// Link this phone's FCM token to the logged-in user, with their type and
package.
  static Future<void> registerDevice(String username,
      {String? userType, String? userPackage}) async {
    if (kIsWeb) return;
    try {
      final token = await _getFcmToken();
      if (token == null) return;
      await _dio.post(
        '/devices',
        data: {
          'token': token,
          'platform': defaultTargetPlatform == TargetPlatform.iOS ? 'ios' :
'android',
          'userType': userType,
          'userPackage': userPackage,
        },
        options: Options(headers: _authHeaders(username)),
      );
      debugPrint('Push: device registered for $username');
    } catch (e) {
      debugPrint('Push: register failed: $e');
    }
  }

  /// Remove the link between this phone and the user (call before logging
out).
  static Future<void> unregisterDevice(String username) async {
    if (kIsWeb) return;
    try {
      final token = await _getFcmToken();
      if (token == null) return;
      await _dio.delete(
        '/devices',
```

```
        data: {'token': token},
        options: Options(headers: _authHeaders(username)),
      );
      debugPrint('Push: device unregistered for $username');
    } catch (e) {
      debugPrint('Push: unregister failed: $e');
    }
  }

  /// Ask the server to send a notification to one user, by username.
  /// Returns a short message to show on screen.
  static Future<String> sendNotification({
    required String fromUsername,
    required String toUsername,
    required String title,
    required String body,
  }) async {
    try {
      final r = await _dio.post(
        '/notifications',
        data: {'toUsername': toUsername, 'title': title, 'body': body},
        options: Options(headers: _authHeaders(fromUsername)),
      );
      final devices = r.data['devices'];
      return devices == 0
          ? 'Saved, but $toUsername has no phone logged in right now'
          : 'Sent to $toUsername';
    } on DioException catch (e) {
      final serverError = e.response?.data is Map ? e.response?.data['error']
: null;
      return 'Failed: ${serverError ?? e.message}';
    }
  }

  /// Ask the server to send a notification to every user of a given type
  /// in a given package (e.g. Consultants in package "3").
  /// Returns a short message to show on screen.
  static Future<String> sendNotificationToType({
    required String fromUsername,
    required String toUserType,
    required String toUserPackage,
    required String title,
    required String body,
```

```
  }) async {
    try {
      final r = await _dio.post(
        '/notifications/by-type',
        data: {
          'toUserType': toUserType,
          'toUserPackage': toUserPackage,
          'title': title,
          'body': body,
        },
        options: Options(headers: _authHeaders(fromUsername)),
      );
      final devices = r.data['devices'];
      return devices == 0
          ? 'No $toUserType in package $toUserPackage has a phone logged in
right now'
          : 'Sent to ${r.data['users']} $toUserType user(s)';
    } on DioException catch (e) {
      final serverError = e.response?.data is Map ? e.response?.data['error']
: null;
      return 'Failed: ${serverError ?? e.message}';
    }
  }

  /// If Firebase rotates this phone's token, register the new one.
  /// The getters return the logged-in user's values, or null when logged
out.
  static void listenTokenRefresh(
    String? Function() getUsername,
    String? Function() getUserType,
    String? Function() getUserPackage,
  ) {
    if (kIsWeb) return;
    FirebaseMessaging.instance.onTokenRefresh.listen((_) {
      final username = getUsername();
      if (username != null) {
        registerDevice(
          username,
          userType: getUserType(),
          userPackage: getUserPackage(),
        );
```

```
      }
    });
  }
}
```

Your editor may underline `onTokenRefresh` with "deprecated". It is only a warning: the app still builds and works, and it only matters when Firebase changes a phone's token, which is rare.

Step 3. `lib/main.dart`: start Firebase

3a. Imports. Add three imports at the very top of the file, above the existing `flutter_neumorphic` import:

```
import 'package:firebase_core/firebase_core.dart';
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/foundation.dart' show kIsWeb;
```

Then add two imports right after the existing `import 'provider/api_provider.dart';` line:

```
import 'firebase_options.dart';
import 'package:tram/noti_services/push_service.dart';
```

3b. Replace `main()`.

Before:

```
void main() async {
  await GetStorage.init();
  await GetStorage.init("Auth");
  runApp(MyApp());
}
```

After:

```
void main() async {
  WidgetsFlutterBinding.ensureInitialized();
  // We only configured Firebase for Android and iPhone, so skip it on
Chrome/web.
  if (!kIsWeb) {
    await Firebase.initializeApp(
      options: DefaultFirebaseOptions.currentPlatform,
```

```
    );
    // iPhone hides pushes while the app is open unless you ask it to show
them.
    await
FirebaseMessaging.instance.setForegroundNotificationPresentationOptions(
      alert: true,
      badge: true,
      sound: true,
    );
  }
  await GetStorage.init();
  await GetStorage.init("Auth");
  // If Firebase changes this phone's token, tell the server again.
  PushService.listenTokenRefresh(
    () => GetStorage("Auth").read("name"),
    () => GetStorage("Auth").read("user_type"),
    () => GetStorage("Auth").read("user_package"),
  );
  runApp(MyApp());
}
```

The `kIsWeb` check matters: `firebase_options.dart` has no web settings, so without it the app would crash on startup in Chrome.

Step 4. `lib/controller/auth/auth_controller.dart`: connect the phone at login, disconnect at logout

4a. Imports. Add these two lines after the existing `import 'package:tram/model/basic/auth_model.dart';` line:

```
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:tram/noti_services/push_service.dart';
```

(`kIsWeb` already comes from the existing `package:flutter/foundation.dart` import.)

4b. In `login()`, add the `if (!kIsWeb)` block between `await box.save();` and the `debugPrint(...)` line. The controller already saves `user_type` and `user_package` from the login response, so the values are ready to send.

Before:

```
    await box.save();
    debugPrint('Login - userType: "${data.userProfileData!.mUserType}"');
  }
```

After:

```
    await box.save();
    if (!kIsWeb) {
      try {
        await FirebaseMessaging.instance.subscribeToTopic('all_users');
      } catch (e) {
        debugPrint('FCM subscribe failed: $e');
      }
      // iPhone needs permission before it gives a token; Android 13+ asks
here too.
      try {
        await FirebaseMessaging.instance.requestPermission();
      } catch (e) {
        debugPrint('FCM permission failed: $e');
      }
      // Link this phone's token to the user who just logged in,
      // with their type (m_userType) and package (m_userPackage).
      await PushService.registerDevice(
        data.userProfileData!.mUsername,
        userType: data.userProfileData!.mUserType,
        userPackage: data.userProfileData!.mUserPackage,
      );
    }
    debugPrint('Login - userType: "${data.userProfileData!.mUserType}"');
  }
```

4c. In `logout()`, add a block between `SmartDialog.showLoading(...)` and `await box.erase();`. It must run **before** the erase, because erasing clears the username.

Before:

```
    SmartDialog.showLoading(
      clickMaskDismiss: false,
    );
    await box.erase();
```

After:

```
    SmartDialog.showLoading(
      clickMaskDismiss: false,
    );
    if (!kIsWeb) {
      try {
        await FirebaseMessaging.instance.unsubscribeFromTopic('all_users');
      } catch (e) {
        debugPrint('FCM unsubscribe failed: $e');
      }
      // Remove the link between this phone and the user.
      final username = accessName();
      if (username != null) {
        await PushService.unregisterDevice(username);
      }
    }
    await box.erase();
```

The `try/catch` blocks and the 5-second timeouts in `PushService` stop a server problem from blocking login or logout.

Step 5. `lib/controller/home/root_home_controller.dart`: the bell's count

Add one line right under `RxInt index = 0.obs;`.

Before:

```
class RootHomeController extends GetxController {
  RxInt index = 0.obs;
  final auth = Get.find<AuthController>();
```

After:

```
class RootHomeController extends GetxController {
  RxInt index = 0.obs;
  RxInt notificationCount = 3.obs; // fixed test number for now
  final auth = Get.find<AuthController>();
```

The `3` is a fixed test value so the badge shows (any number above 0 works). Later it should come from the `notifications` table (count of unread rows).

Step 6. `lib/constant/routes.dart`: register the page

6a. Import. Add this line with the other imports at the top, for example right after `import '../screen/login/login_screen.dart';`:

```
import 'package:tram/screen/notification/notification_screen.dart';
```

6b. Route. Add a route at the very bottom of the `routes` list, just before the closing `];`. After the commented-out blocks is fine.

Before:

```
  // GetPage(
  //   name: '/all_gis_map',
  //   page: () => const AllGisMapScreen(),
  //   binding: AllGisMapBinding(),
  // ),
];
```

After:

```
  // GetPage(
  //   name: '/all_gis_map',
  //   page: () => const AllGisMapScreen(),
  //   binding: AllGisMapBinding(),
  // ),

  GetPage(
    name: '/notifications',
    page: () => const NotificationScreen(),
  ),
];
```

The page has no controller, so it needs no binding.

Step 7. New file `lib/screen/notification/notification_screen.dart`

Create the folder `lib/screen/notification/` and put this file in it. It has the **submit form b** button, plus two test buttons (ask for permission, show this phone's token). The buttons only appear on the phone app, not on web.

When you tap **submit form b**, the page reads your username and your package from the login data, and asks the server to notify every `Consultant` in that same package.

```
import 'package:firebase_messaging/firebase_messaging.dart';
import 'package:flutter/foundation.dart' show kIsWeb;
import 'package:flutter/services.dart';
import 'package:flutter_neumorphic/flutter_neumorphic.dart';
import 'package:get/get.dart';

import '../../components/Neumorphic/navbar.dart';

import 'package:tram/controller/auth/auth_controller.dart';
import 'package:tram/noti_services/push_service.dart';

/// Notifications page.
/// It has a "submit form b" button that notifies the Consultants in the same
package,
/// plus test buttons to ask permission and show this phone's FCM token.
/// The real list of notifications will go in the body later.
class NotificationScreen extends StatefulWidget {
  const NotificationScreen({super.key});

  @override
  State<NotificationScreen> createState() => _NotificationScreenState();
}

class _NotificationScreenState extends State<NotificationScreen> {
  String _status = 'Permission: not asked yet';
  String? _token;

  String _sendResult = '';
  bool _sending = false;

  /// "submit form b": notify every Consultant in the same package as the
submitter.
  Future<void> _submitFormB() async {
    final auth = Get.find<AuthController>();
    final from = auth.accessName();
    final package = auth.getUserPackage();
    if (from == null || package == null || package.isEmpty) {
      setState(() => _sendResult = 'Your account has no package, cannot
send');
      return;
    }

    setState(() {
```

```
      _sending = true;
      _sendResult = '';
    });
    final result = await PushService.sendNotificationToType(
      fromUsername: from,
      toUserType: 'Consultant',
      toUserPackage: package,
      title: 'Form B submitted',
      body: '$from submitted Form B (Package $package)',
    );
    if (!mounted) return;
    setState(() {
      _sending = false;
      _sendResult = result;
    });
  }

  Future<void> _requestPermission() async {
    final settings = await FirebaseMessaging.instance.requestPermission();
    setState(() {
      _status = 'Permission: ${settings.authorizationStatus.name}';
    });
  }

  Future<void> _getToken() async {
    final token = await FirebaseMessaging.instance.getToken();
    setState(() => _token = token);
  }

  Future<void> _copyToken() async {
    if (_token == null) return;
    await Clipboard.setData(ClipboardData(text: _token!));
    Get.snackbar(
      'Copied',
      'FCM token copied to clipboard',
      colorText: Colors.black,
      margin: const EdgeInsets.all(10),
      duration: const Duration(seconds: 2),
    );
  }

  @override
  Widget build(BuildContext context) {
```

```
    return Scaffold(
      appBar: const NeumorphicNavBar(
        title: 'Notifications',
        showDrawer: false, // shows a back arrow instead of the menu icon
        color: Color(0xfff3f8fe),
      ),
      backgroundColor: const Color(0xfff3f8fe),
      body: SingleChildScrollView(
        padding: const EdgeInsets.all(20),
        child: Column(
          children: [
            const Text('No notifications yet'),
            const SizedBox(height: 30),
            if (kIsWeb)
              const Text('FCM test buttons only work on the phone app.')
            else ...[
              // Notify the Consultants in the same package
              NeumorphicButton(
                onPressed: _sending ? null : _submitFormB,
                child: Text(_sending ? 'Sending...' : 'submit form b'),
              ),
              const SizedBox(height: 8),
              Text(_sendResult),
              const SizedBox(height: 30),

              // Test buttons
              Text(_status),
              const SizedBox(height: 12),
              NeumorphicButton(
                onPressed: _requestPermission,
                child: const Text('1. Allow notifications'),
              ),
              const SizedBox(height: 20),
              NeumorphicButton(
                onPressed: _getToken,
                child: const Text('2. Get FCM token'),
              ),
              const SizedBox(height: 12),
              if (_token != null) ...[
                SelectableText(
                  _token!,
                  style: const TextStyle(fontSize: 12),
                ),
```

```
                const SizedBox(height: 12),
                NeumorphicButton(
                  onPressed: _copyToken,
                  child: const Text('Copy token'),
                ),
              ],
            ],
          ],
        ),
      ),
    );
  }
}
```

Optional clean-up later. Permission is now asked at login, so the two test buttons are no longer needed for normal use. Keep them while you test: a denied permission cannot be asked again, and the status text shows what happened.

Step 8. `lib/screen/home/root_home_screen.dart`: the bell in the header

The header's `actions` list (the right side, between the logo and the logout button) holds only the logout button. Replace it.

Before:

```
          actions: [
            NeumorphicButton(
              onPressed: () {
                controller.logout();
              },
              child: const Icon(Icons.logout, size: 20),
            )
          ],
```

After:

```
          actions: [
            if (controller.index.value == 5)
              NeumorphicButton(
                onPressed: () {
                  Get.toNamed('/notifications');
                },
                child: Badge(
                  isLabelVisible: controller.notificationCount.value > 0,
                  label: Text('${controller.notificationCount.value}'),
                  child: const Icon(Icons.notifications_none, size: 20),
                ),
              ),
            NeumorphicButton(
              onPressed: () {
                controller.logout();
              },
              child: const Icon(Icons.logout, size: 20),
            )
          ],
```

How it works:

- `controller.index.value == 5` is the Form Depot tab, so the bell only shows there. The same file already uses this check for the `edit_note` icon.
- The whole screen is wrapped in `Obx(...)`, so the badge number updates by itself when `notificationCount` changes.
- The badge hides when the count is 0.
- Tapping the bell calls `Get.toNamed('/notifications')`, which opens the page from Step 7 through the route from Step 6.

Now run `flutter analyze` in the app's root folder. All eight steps are done, so it should report no errors. The `onTokenRefresh` deprecation warning from Step 2 may still show: that is fine.

Part E: Build, install and test on Android

For the full test you need two Android phones: phone A submits, phone B receives. Both phones and your PC must be on the same Wi-Fi. With only one phone, see the single-phone test at the end of E3.

E1. Get everything running

1. Start the database if it is not running: `docker compose up -d` in `fcm_server`.
2. Start the server: `node server.js` in `fcm_server`. Leave this window open and watch it during the test.
3. Run `ipconfig`, find your PC's **IPv4 Address**, and make sure `lib/constant/fcm_config.dart` (Part D, step 1) uses it.
4. On each phone, open the browser and go to `http://<your-pc-ip>:4000/health`. You should see `{"ok":true,...}`. If not, the phone cannot reach the server: check the Wi-Fi, the IP address and the Windows firewall (allow Private networks).

E2. Build and install the app

In the app's root folder:

```
flutter analyze
flutter build apk --debug
```

`flutter analyze` should report no errors. The first build with Firebase takes several minutes. Then send `build\app\outputs\flutter-apk\app-debug.apk` to both phones, allow installs from that source, and install it. Installing over an older version keeps the phone's Firebase token and login. Uninstalling first gives the phone a new token.

E3. Test with two phones

The test accounts in our project: `user.level4` is a **Consultant** in package **3**. Use any other account in package 3 as the submitter.

1. **Log in.** Phone B logs in as `user.level4` (Consultant, package 3). Phone A logs in as another user in package 3. The server window prints `Registered device for <username>` once per login. On Android 13 or newer, phone B shows an "Allow notifications" popup at login: tap **Allow**.
2. **Check the database.** Run:

   ```
   docker exec -it trams_noti_db psql -U root -d trams_noti_db -c "SELECT
   username, platform, user_type, user_package, left(fcm_token, 25) AS
   token_start FROM device_tokens;"
   ```

   You should see one row per logged-in phone. `user.level4` shows `android`, `Consultant` and `3`.

3. **Prepare phone B.** Open the Form Depot tab. The bell with a badge sits between the logo and the logout button. Other tabs show no bell. If the permission popup did not appear at login, tap the bell, then **1. Allow notifications**, and choose Allow. The text should read `Permission: authorized`.
4. **Swipe the app away on phone B.** Android shows nothing for a push while the app is open on screen.
5. **Submit from phone A.** Open the Form Depot tab, tap the bell, and tap **submit form b**.
6. **Check the result.** Phone B buzzes within a few seconds with "Form B submitted". Phone A says `Sent to 1 Consultant user(s)`. The server window prints `Notification by type "Consultant" pkg 3: ... (devices 1, sent 1, removed 0)`.

If phone A says `No Consultant in package 3 has a phone logged in right now`, phone B is not registered: log out and in again on phone B.

Single-phone test. Log in as `user.level4` on your only phone, tap **submit form b**, then swipe the app away or lock the phone. The push arrives on your own phone, because you are a Consultant in package 3 yourself. The server sends to the submitter too.

E4. Test the package and type rules

The notification must reach only Consultants in the submitter's package. Repeat E3 steps 5 and 6 with these cases:

Case	Expected
Submitter in package 3, Consultant logged in with package 3	The Consultant's phone buzzes
Submitter in package 4, Consultant logged in with package 3	The Consultant does not buzz. The submitter sees `No Consultant in package 4 has a phone logged in right now`
Submitter in package 3, a Service Provider in package 3 logged in	The Service Provider does not buzz, because the type is not `Consultant`

Check saved rows with `SELECT to_username, from_username, title FROM notifications ORDER BY id DESC;`. Each recipient has one row.

E5. Test two accounts on one phone

On phone B, log out, then log in as a third user. Run the `SELECT` from E3 step 2 again. The row for that token now shows the new username, type and package, and there is still only one row for the phone. Submit again as a user who would have reached the old account: phone B no longer buzzes, because the link was removed at logout.

E6. Test the cleanup of dead tokens (optional)

Register a fake token for a Consultant in the same package, then submit:

```
Invoke-RestMethod -Method Post -Uri http://localhost:4000/devices -Headers @{
"X-Test-User" = "user.level4" } -ContentType "application/json" -Body
'{"token":"fake-dead-token","userType":"Consultant","userPackage":"3"}'
```

The next submit reports `devices 2, sent 1, removed 1`, and the fake row is deleted.

Part F: iPhone setup (Mac, Xcode, Apple, Firebase)

The server and the database need no changes for the iPhone: Firebase delivers to iPhones through Apple's push service (APNs), and your server asks Firebase exactly as before. The work is on the Mac, in Apple's developer site and in the Firebase console. The Dart code for the iPhone is already in Part D. Do the steps in order. These steps use Terminal on the Mac.

You need a Mac, a **paid Apple Developer account** (a free account cannot create the push key in F5) and a real iPhone.

F1. Get the project onto the Mac

Commit and push the project from the PC with all of Part D done, then clone it on the Mac:

```
git clone <your-repo-url>
cd <your-project-folder>
```

The Mac needs Flutter, Node.js and Git installed. Check with `flutter --version`, `node -v` and `git --version`. Then open `lib/constant/fcm_config.dart` and make sure the IP is your PC's current IPv4 address. The iPhone must be on the same Wi-Fi as the PC.

F2. Install Xcode

1. Check your macOS version: `sw_vers`. Each Xcode needs a minimum macOS.

2. Install **Xcodes.app** (it downloads and manages Xcode versions), open it and sign in with your Apple ID. Installing Xcode from the Mac App Store also works.
3. Pick the newest **stable** Xcode that your macOS can run. Skip beta and release-candidate versions: Flutter and its plugins often do not support them yet. We used Xcode 26.6 on macOS 26.7. A newer pre-release (Xcode 27) was also on the list and we did not use it.
4. Click **Install**. It asks for your Mac password and takes a long while. In the "components" window, keep **macOS** and **iOS** ticked and untick the others, including the code completion model.
5. When it finishes, click **Active** on that version in Xcodes.

Then point Terminal at it. First list the installed Xcodes to see the exact name:

```
ls -d /Applications/Xcode*
```

Ours was `/Applications/Xcode-26.6.0.app` (note the extra `.0`). Use your exact name:

```
sudo xcode-select --switch /Applications/Xcode-26.6.0.app/Contents/Developer
xcodebuild -version
sudo xcodebuild -runFirstLaunch
```

`xcodebuild -version` must print the version you installed. If it prints something else (we first got a pre-release 27), the `xcode-select` command did not take effect: check the path from `ls`.

Check that the iPhone SDK is there:

```
xcodebuild -showsdks
```

You need an `iOS 26.x` entry under "iOS SDKs". If `xcodebuild -downloadPlatform iOS` says "not available for download", ignore it as long as the SDK is listed: that download is only for the simulator, which you do not need. Then check Flutter:

```
flutter doctor
```

If CocoaPods is missing, install it with `brew install cocoapods`.

F3. Install the Firebase tools on the Mac

```
npm install -g firebase-tools
firebase login
dart pub global activate flutterfire_cli
```

`firebase login` opens a browser: sign in with the Google account that owns the Firebase project. After the third command Dart warns that `~/.pub-cache/bin` is not on your PATH, and `flutterfire` will say "command not found". Fix it for the current window and for all future windows:

```
export PATH="$PATH":"$HOME/.pub-cache/bin"
echo 'export PATH="$PATH":"$HOME/.pub-cache/bin"' >> ~/.zshrc
source ~/.zshrc
flutterfire --version
```

F4. Register the iPhone app in Firebase

In the project folder:

```
flutter pub get
flutterfire configure --project=trams-test-f2a06 --platforms=android,ios
```

Keep `android` in the list, or Android's settings can be dropped from `lib/firebase_options.dart`. When it asks to overwrite `lib/firebase_options.dart`, answer yes. The command registers an iOS app with the bundle ID that is currently in the Xcode project (ours was already `com.ai.trams`, the same as Android), regenerates `lib/firebase_options.dart` with iOS values, and creates `ios/Runner/GoogleService-Info.plist`. Check the file exists:

```
ls ios/Runner/GoogleService-Info.plist
```

If your bundle ID is not what you want, change it in Xcode first (step F8), then run this command. Commit the regenerated `lib/firebase_options.dart`. A `tram (macos)` app may also show in the Firebase console from an earlier run without `--platforms`. It is harmless and can be deleted under Project settings.

F5. Create the APNs key (Apple Developer site)

The APNs key lets Firebase send to Apple on your behalf.

1. Go to developer.apple.com, click **Account** (top right) and sign in with the Apple ID of your organisation's Developer account.
2. Open **Certificates, Identifiers & Profiles**, then **Keys** in the left menu, then the blue **+** button.
3. Enter a **Key Name** (letters and numbers only, for example `tramsapns`) and tick **Apple Push Notifications service (APNs)**.
4. Click **Configure** next to APNs. **Continue** stays greyed out until you do this.
5. Set **Environment** to **Sandbox & Production** if that option is listed. If you only see separate options, choose **Sandbox**, which is enough for testing on your own iPhone. You cannot change this after saving.
6. Set **Key Restriction** to **Team Scoped (All Topics)**, so the key works for every app in your team without listing the bundle ID.
7. Click **Save**, then **Continue**, then **Register**.
8. Click **Download**. This saves a `.p8` file, and **you can only download it once**. Keep it safe, and never commit it to git, put it in the app or send it to anyone.

Write down two values:

- **Key ID**: 10 characters. It is on the key's page and also in the file name `AuthKey_XXXXXXXXXX.p8`.
- **Team ID**: 10 characters. It is shown at the top right of the Apple Developer page, next to your team's name (ours was `6J7PDUK2D9`).

If the **+** button is greyed out, your Apple ID's role does not allow creating keys: ask the Account Holder or an Admin. Apple allows only a few APNs keys per account, so if the limit is reached, ask your team whether an existing key can be used.

F6. Upload the key to Firebase

1. In the Firebase console, click **Settings** (gear icon), then **Project settings**, then the **Cloud Messaging** tab.
2. Scroll to **Apple app configuration**. Select the **iOS** app (`com.ai.trams`), not the macOS one.
3. Under **APNs Authentication Key** (not "APNs Certificates", which is the older method), click **Upload** on the **development** row first. Builds you run from Xcode or `flutter run` use Apple's sandbox, which is what this row is for.
4. Choose the `.p8` file, enter the **Key ID** and the **Team ID**, and click **Upload**. The row now shows the file, Key ID and Team ID.

5. If you chose **Sandbox & Production** in F5, upload the same file to the **production** row as well. You do not need it for testing today, but TestFlight and App Store builds need it.

F7. Install the iOS dependencies

```
flutter pub get
cd ios
pod install
cd ..
```

CocoaPods may print a warning that it "did not set the base configuration" for the profile build. You can ignore it: you use the debug and release builds. Only a few pods are installed (ours: Flutter, OrderedSet and an in-app web view plugin). That is normal: Xcode fetches the Firebase packages by itself the first time you open the project.

F8. Open the project in Xcode and set up signing

Open the **workspace** (not the `.xcodeproj`) in the Xcode you installed. Plain `open ios/Runner.xcworkspace` may launch your code editor instead of Xcode, so name the app:

```
open -a /Applications/Xcode-26.6.0.app ios/Runner.xcworkspace
```

Use your exact Xcode name from F2. At the top of the window, wait until the spinner ("Updating GoogleUtilities..." or similar) finishes. Then:

1. In the left sidebar, click the blue **Runner** project at the top. Do not click **Pods**. If the sidebar is hidden, press `Cmd + 0`.
2. In the middle panel, under **TARGETS**, click **Runner**.
3. Click the **Signing & Capabilities** tab.
4. Keep **Automatically manage signing** ticked. Set **Team** to your organisation's account. If the list is empty, go to **Xcode, then Settings, then Accounts**, click **+** and sign in with your Apple ID.
5. Check that **Bundle Identifier** is `com.ai.trams`, exactly the same as in Firebase.
6. Click **+ Capability**, search for **Push Notifications** and double-click it.
7. Click **+ Capability** again, add **Background Modes**, and tick **Remote notifications** in its list.

The **Provisioning Profile** line should say "Xcode Managed Profile" and no red errors should show. Existing capabilities, such as Location, can stay. A yellow warning is fine.

F9. Edit `ios/Runner/Info.plist`

Open `ios/Runner/Info.plist` in your code editor. Your server is `http://<pc-ip>:4000`, which iOS blocks by default. This is the iPhone version of Android's `usesCleartextTraffic` (Part A, step A7). Search the file first: if `NSAppTransportSecurity` or `NSLocalNetworkUsageDescription` already exist, do not add them twice. Otherwise add these two entries inside the main `<dict>`, for example right after the other `NS...` entries:

```
        <key>NSAppTransportSecurity</key>
        <dict>
                <key>NSAllowsArbitraryLoads</key>
                <true/>
        </dict>
        <key>NSLocalNetworkUsageDescription</key>
        <string>Needed to reach the notification server during testing.
</string>
```

The first entry lets the app call a plain `http` server. The second makes iOS show the "local network" permission prompt, which it needs for an address like `192.168.x.x`. Also check that the file contains this block (Xcode adds it when you tick **Remote notifications** in F8):

```
        <key>UIBackgroundModes</key>
        <array>
                <string>remote-notification</string>
        </array>
```

Both `http` entries are for testing only. A production server uses `https`, and then `NSAllowsArbitraryLoads` is removed.

Part G: Build and test on the iPhone

An Android `.apk` file cannot be installed on an iPhone. For testing, build straight onto the phone with a cable.

G1. Connect the iPhone

1. Connect the iPhone to the Mac with a cable, unlock it and tap **Trust** when asked.

2. Turn on **Developer Mode** on the iPhone: **Settings, then Privacy & Security, then Developer Mode**. The phone restarts. The option appears after the phone has been connected to a Mac running Xcode.
3. Find the iPhone's ID:

   ```
   flutter devices
   ```

   Your iPhone is listed with its name and a long ID in the middle, for example `00008110-001A2B3C4D5E6F78`. If it is missing, check that it is unlocked, trusted and in Developer Mode, then run the command again.

G2. Build and run on the iPhone

```
flutter run --release -d <your-iphone-id>
```

Use `--release` so the app keeps working after you unplug the phone. On recent iOS versions, a debug build installed this way cannot be opened from the home screen without the debugger attached. A release build signed with your team's development profile still uses Apple's sandbox push system, which is what the development key from F6 covers.

The first build takes several minutes. If the iPhone says "Untrusted Developer", go to **Settings, then General, then VPN & Device Management**, and trust your profile.

G3. Test

1. **Log in as a Consultant** on the iPhone, for example `user.level4` (package 3). Tap **Allow** on the notification popup and on the "local network" popup.
2. **Check the database.** The iPhone row must show `ios`:

   ```
   docker exec -it trams_noti_db psql -U root -d trams_noti_db -c "SELECT
   username, platform, user_type, user_package FROM device_tokens;"
   ```

3. **Put the iPhone in the background** or lock it. (With the app open on screen, the iPhone shows the banner as well, because of the foreground setting in Part D, step 3.)
4. **Submit from another phone.** On an Android phone logged in as another user in package 3, open the Form Depot tab, tap the bell, and tap **submit form b**.
5. **Check the result.** The iPhone shows "Form B submitted". The server window prints `Notification by type "Consultant" pkg 3: ... (devices 1, sent 1, removed 0)`.

Also test the other way round: log the Android phone in as a Consultant in package 3 and submit from the iPhone as another user in package 3. And repeat the package and type cases from E4 with the iPhone as the Consultant.

The PC running `fcm_server`, the Android phone and the iPhone must all be on the same Wi-Fi, and `lib/constant/fcm_config.dart` must hold the PC's current IP.

G4. Installing without a cable (later)

- **TestFlight:** upload the build through App Store Connect and install it from the TestFlight app. This uses Apple's production push system, so upload the `.p8` key to the **production** row in Firebase too (F6, step 5).
- **Ad hoc `.ipa`:** build with `flutter build ipa` and install it through Xcode's Devices window. The iPhone's UDID must be registered in your Apple Developer account first.

If something goes wrong

Server and database

Symptom	Likely cause	Fix
Server: `require is not defined in ES module scope`	A file uses `require`, but `package.json` has `"type": "module"`	Use `import` as in Part C; keep the `.js` ending on local imports
Server: `Cannot read properties of undefined (reading 'cert')`	Old `firebase-admin` import style	Use the imports in C6
`/health` returns an error	`DATABASE_URL` wrong (check the port is `5004`), or the database is not running	Check `.env` against Part B; run `docker compose up -d`
Server: `column "user_type" of relation "device_tokens" does not exist`	The table was created before the two new columns existed	Run the two `ALTER TABLE` commands in B2
Phone's browser cannot open `http://<pc-ip>:4000/health`	Wrong IP, different Wi-Fi, or firewall	E1 step 4; check `ipconfig` and allow Private networks in the Windows firewall

Symptom	Likely cause	Fix
Error `toUserType, toUserPackage and title required`	The app did not send the package: an old build, or the account has no package	Rebuild the app with Part D; check the login response has `m_userPackage`

App

Symptom	Likely cause	Fix
App crashes on start in Chrome	Firebase started on web without web settings	Keep the `kIsWeb` check in `main()` (Part D, step 3)
"Target of URI doesn't exist: ...push_service.dart"	The file is not in `lib/noti_services/`, or the import path differs from the folder name	Move the file to `lib/noti_services/push_service.dart`; make `main.dart`, `auth_controller.dart` and `notification_screen.dart` import exactly that path
Build fails with "Java heap space"	Gradle's default memory is too small	A6
`firebase` or `flutterfire` "not recognized" (PC)	Terminal opened before install, or the Dart tools folder is not on PATH	Reopen the terminal; add the Pub Cache folder to PATH (A2)
`onTokenRefresh` is deprecated (yellow underline)	Newer `firebase_messaging` marks it deprecated	Only a warning: the app still builds and works. Ignore it for now
Submit says "Your account has no package, cannot send"	`user_package` is empty in the login data	Log out and in again; check the login response has `m_userPackage`
App says `Failed: ... connection ...`	Wrong IP in `fcm_config.dart`, different Wi-Fi, or plain `http` blocked	Fix the address in `fcm_config.dart`; Android: check A7; iPhone: check F9 and the "Local Network" switch for the app in the iPhone's Settings; rebuild

Delivery

Symptom	Likely cause	Fix
`No Consultant in package X has a phone logged in right now`, or `devices 0`	No phone is registered with that exact type and package. The Consultant logged in before this feature was added, logged out, or the package differs	Run the `SELECT` from E3 step 2. `user_type` must be exactly `Consultant` (case matters) and `user_package` equal to the submitter's. Log out and in again on the Consultant's phone
`sent 1` but no notification on the phone	Permission off, Android app open on screen, or battery / Do Not Disturb settings	Allow notifications; swipe the app away; set battery use to Unrestricted. A reinstall resets the permission
`removed 1` or `NotRegistered`	The token is dead, usually after reinstalling the app	Log in again on that phone to register its new token

iPhone and Mac

Symptom	Likely cause	Fix
`flutterfire: command not found` (Mac)	`~/.pub-cache/bin` is not on PATH	The three `export` / `source` commands in F3
`Unable to find application named 'Xcode'`	Only **Xcodes.app** is installed. It is a helper, not Xcode itself	Install an Xcode version from Xcodes (F2)
`xcode-select: error: invalid developer directory`	The Xcode folder name is not what you typed (for example `Xcode-26.6.0.app`)	Run `ls -d /Applications/Xcode*` and use that exact name
`xcodebuild -version` shows a pre-release Xcode	The wrong Xcode is active	Switch with `xcode-select` to the stable one (F2)

Symptom	Likely cause	Fix
`open ios/Runner.xcworkspace` opens your code editor	The editor is the default app for that file type	Use `open -a /Applications/<your Xcode>.app ios/Runner.xcworkspace`
Xcode shows a list of `Pods` targets and no Signing tab	The **Pods** project is selected	Click the blue **Runner** project at the top of the sidebar, then the **Runner** target (F8)
iPhone not listed by `flutter devices`	Phone locked, not trusted, or Developer Mode off	Unlock, tap Trust, turn on Developer Mode (G1)
"Untrusted Developer" on the iPhone	The developer profile is not trusted yet	Settings, General, VPN & Device Management, trust your profile
No `ios` row appears in `device_tokens` after login	Notification permission refused, the Push Notifications capability is missing, or you are on the simulator	Check F8 capabilities; delete the app, run it again and tap Allow; use a real iPhone
The `ios` row is there, `sent 1`, but no push on the iPhone	The APNs key is missing or wrong in Firebase (wrong Key ID or Team ID, wrong app selected, or uploaded to the wrong row)	Redo F6 for the iOS app `com.ai.trams`; development row for builds from Xcode / `flutter run`
`pod install` prints "did not set the base configuration"	CocoaPods warning about the profile build	Ignore it (F7)

Not done yet

This guide gets you to "a user taps **submit form b** and the Consultants in the same package get a push, on Android and iPhone". These parts are still missing, in the order we plan to do them:

1. **Bell count and list from the database.** `notificationCount` is a fixed `3`, and the notifications page has no list. The `notifications` table already stores everything needed: add an endpoint for the unread count, one for the list, and one to mark a notification as read.
2. **Connect the button to the real Form B submit. submit form b** on the Notifications page is a test button. The real notification should be sent from the code that submits a Form B, using the same call (`PushService.sendNotificationToType`).
3. **Pushes while the app is open (Android) and tapping a notification.** Android shows nothing on its own if the app is on screen. A `FirebaseMessaging.onMessage` listener should refresh the bell and the list, and tapping a notification should open `/notifications`. (The iPhone already shows a banner in the foreground, because of the setting in Part D, step 3.)
4. **Real authentication.** `requireUser` in `auth.js` and `_authHeaders` in `push_service.dart` are test stand-ins that trust a username header. In production, replace both with the real access token check. At the same time, make the server read the submitter's **package** from the verified login instead of trusting `toUserPackage` from the request.
5. **Choose who receives it.** Right now the sender is also notified if they are a Consultant in that package. Decide whether to exclude the sender, and whether to pick recipients from a list instead of by type.
6. **Production setup.** Use your organisation's own Firebase project (repeat A3, F4 and C1 against it), run the server over `https` on a real domain, remove `usesCleartextTraffic` (Android) and `NSAllowsArbitraryLoads` (iPhone `Info.plist`), upload the APNs key to the **production** row in Firebase, and keep `serviceAccountKey.json`, `.env` and the `.p8` key out of git. Distribute iPhone builds through TestFlight.
7. **Optional clean-up.** Remove the two test buttons on the Notifications page (**1. Allow notifications** and **2. Get FCM token**) once both platforms work, and replace the deprecated `onTokenRefresh` listener when you update `firebase_messaging`.